

Vave Geospatial Risk Aggregation Pipeline - Overview

Executive Summary

The Vave pipeline is a **batch-driven, medallion-architecture data pipeline** that processes geospatial event data with risk scores, enriches it with H3 hexagonal indexing, and produces daily aggregated analytics for visualization.

Purpose: Aggregate and visualize risk events by geographic location (H3 cells) and time (daily granularity) for the London area.

Geographic Coverage: London, UK (lat: 51.47-51.54, lon: -0.18 to -0.07)

H3 Resolution: 8 (~0.46 km hexagon edge)

Time Range: April-May 2026 (sample data)

Layers:

- **Bronze:** Raw CSV data with type casting
- **Silver:** Enriched with H3 indexing, date partitioning
- **Gold:** Daily aggregations by H3 cell/zone for analytics

Executive Summary

The Vave pipeline is a **batch-driven, medallion-architecture data pipeline** that processes geospatial event data with risk scores, enriches it with H3 hexagonal indexing, and produces daily aggregated analytics for visualization.

Purpose: Aggregate and visualize risk events by geographic location (H3 cells) and time (daily granularity) for the London area.

Geographic Coverage: London, UK (lat: 51.47-51.54, lon: -0.18 to -0.07)

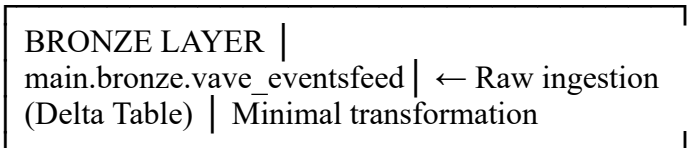
H3 Resolution: 8 (~0.46 km hexagon edge)

Time Range: April-May 2026 (sample data)

Pipeline Architecture - Medallion Pattern

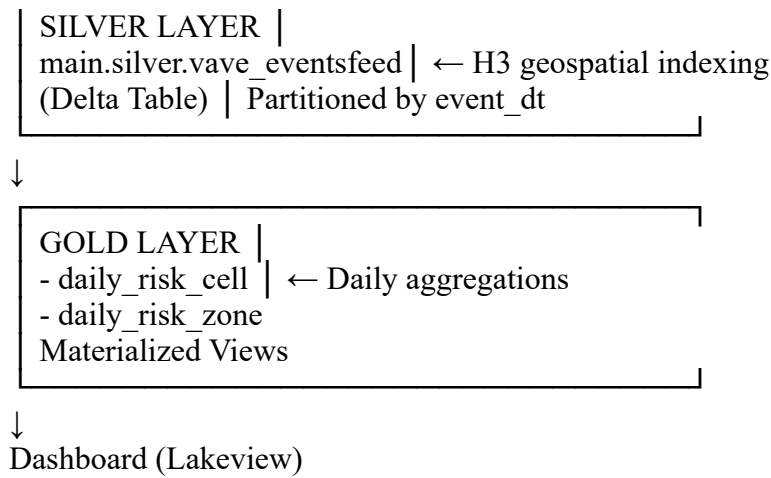
CSV Files (Unity Catalog Volume)

↓



↓





Layers:

- **Bronze:** Raw CSV data with type casting
- **Silver:** Enriched with H3 indexing, date partitioning
- **Gold:** Daily aggregations by H3 cell/zone for analytics

Bronze Layer: Raw Data Landing

Table: `main.bronze.vave_eventsfeed`

Schema: | Column | Type | Description | |-----|-----|-----| | event_id | STRING | Unique event identifier | | event_timestamp | TIMESTAMP | Event occurrence time | | latitude | DOUBLE | Geographic latitude | | longitude | DOUBLE | Geographic longitude | | risk_score | DOUBLE | Risk score (0-100) | | source_file_name | STRING | Source CSV file path | | ingestion_ts | TIMESTAMP | Pipeline ingestion timestamp | | dbt_run_id | STRING | Batch/run identifier |

Transformations:

- Type casting (STRING → TIMESTAMP, DOUBLE)
- File metadata capture via `_metadata.file_path`
- Ingestion timestamp addition

Control: Tracks processed files in `main.control.processed_files` (idempotent)

Silver Layer: Enriched with H3 Indexing

Table: `main.silver.vave_eventsfeed`

Additional Columns: | Column | Type | Description | |-----|-----|-----| | h3_ix | STRING | H3 geospatial index (resolution 8) | | event_dt | DATE | Event date (partition key) |

Transformations:

- **H3 Indexing:** `h3_latlng_to_cell_string(latitude, longitude, 8)`
 - Resolution 8 $\approx$ 0.46 km hexagon edge
 - Enables spatial aggregation

- **Date Extraction:** DATE(event_timestamp) for partitioning
- **Deduplication:** Prevents duplicate event_ids

Partitioning: By event_dt for query performance on date ranges

Why H3?

- Uniform hexagonal grid (better than squares for neighbor analysis)
- Hierarchical (multi-resolution support)
- Global coverage (works anywhere on Earth)

Gold Layer: Daily Aggregations

View 1: main.gold.daily_risk_cell

Purpose: Daily risk aggregation by H3 cell

Schema: | Column | Type | Description | |-----|-----|-----| | event_dt | DATE | Event date | | h3_ix | STRING | H3 cell identifier | | event_count | LONG | Number of events | | total_risk_score | DOUBLE | Sum of risk scores | | avg_risk_score | DOUBLE | Average risk score |

View 2: main.gold.daily_risk_zone

Purpose: Daily risk aggregation by zone (cell + neighbors)

Schema: | Column | Type | Description | |-----|-----|-----| | event_dt | DATE | Event date | | zone_id | STRING | Zone identifier | | contributing_cells | LONG | Cells in zone | | event_count | LONG | Total events | | total_risk_score | DOUBLE | Sum of risk scores | | avg_cell_risk_score | DOUBLE | Avg risk per cell |

Type: Materialized Views (pre-computed, incrementally refreshable on serverless)

ASSUMPTION 1: Pipeline Cadence - Daily Batch

Current Design: Daily batch processing (e.g., 2 AM schedule)

Rationale:

- Sample data shows daily event patterns
- Dashboard granularity is daily (event_dt)
- Cost-effective for moderate data volumes
- Simple orchestration (single daily job)

When to Reconsider: | Requirement | Recommendation | |-----|-----| | Latency < 1 hour | Migrate to **streaming pipeline** | | Latency < 15 min | Implement **micro-batching** (every 5-15 min) | | Real-time alerts | Use **Spark Structured Streaming** |

Streaming Alternative:

ASSUMPTION 2: Data Volume - Thousands of Rows

Current Scale: ~190 events (sample data)

Design Implications:

- Single-node processing sufficient
- No horizontal partitioning required
- Materialized views refresh quickly (<1 min)

Scaling Recommendations:

Daily Volume	Action Required
< 100K rows	Current design OK
100K - 1M rows	Enable auto-optimization, consider hourly partitioning
1M - 10M rows	Streaming architecture, Z-ordering on h3_ix, larger cluster
> 10M rows	Horizontal partitioning (by region/date), liquid clustering

Performance Optimization (if scaling):

```
%sql
-- Z-ordering for spatial queries
OPTIMIZE main.silver.vave_eventsfeed ZORDER BY (h3_ix);

-- Liquid clustering (high cardinality)
ALTER TABLE main.silver.vave_eventsfeed
CLUSTER BY (h3_ix, event_dt);
```

ASSUMPTION 3: Batch vs Streaming - Batch Processing

Current Design: Batch processing with file-based ingestion

Batch Characteristics:

-  Simple orchestration (Databricks Jobs, dbt)
-  Cost-effective (compute only when needed)
-  Easy to debug and replay
-  Higher latency (daily updates)
-  Not suitable for real-time use cases

When to Switch to Streaming:

Scenario	Trigger Point
Latency requirements	Need data within 1 hour
Continuous data flow	Files arrive every few minutes

Scenario	Trigger Point
High file volume	> 1000 files/day
Event-driven processing	Immediate response to file arrival

Hybrid Approach (Best of Both):

- **Streaming** for Bronze → Silver (low latency ingestion)
- **Batch** for Silver → Gold (scheduled aggregations)

ASSUMPTION 4: Time Granularity - Daily

Current Design: Daily aggregations (event_dt)

Rationale:

- Dashboard focuses on daily trends
- Sample data shows daily patterns
- Simplifies partitioning and queries

Alternative Granularities:

Granularity	Use Case	SQL Example
Hourly	Intraday monitoring	DATE_TRUNC('hour', event_timestamp)
15-minute	Near real-time	Windowed streaming aggregation
Weekly	Executive reporting	DATE_TRUNC('week', event_timestamp)
Monthly	Long-term trends	DATE_TRUNC('month', event_timestamp)

Trade-offs:

- **Finer granularity** (hourly/minute):
 - More data points
 - Slower dashboard queries
 - Higher storage costs
- **Coarser granularity** (weekly/monthly):
 - Less granular insights
 - Faster queries
 - Lower storage costs

ASSUMPTION 5: Geographic Coverage - London Area

Current Coverage: London, UK (lat: 51.47-51.54, lon: -0.18 to -0.07)

H3 Resolution: 8 (~0.46 km hexagon edge)

H3 Resolution Trade-offs:

Resolution	Hexagon Edge	Use Case
6	~3.23 km	City-level
7	~1.22 km	District-level
8	~0.46 km	Current - Neighborhood
9	~0.17 km	Street-level
10	~0.06 km	Building-level

Considerations:

- **Coarser resolution** (6-7): Fewer cells, faster aggregation, less granular
- **Finer resolution** (9-10): More cells, slower queries, very granular

Scaling to Multiple Regions:

- Pipeline is **region-agnostic** (H3 works globally)
- No hardcoded lat/lon filters
- To support multiple cities: Add region column in Bronze layer

Performance Optimizations

Current Optimizations

1. **Silver Partitioning:** By event_dt for date-range queries
2. **Materialized Views:** Pre-computed Gold aggregations
3. **Delta Format:** ACID transactions, time travel, schema evolution
4. **Idempotent Processing:** Control table prevents duplicate work

Future Optimizations (If Volume Increases)

```
%sql
ALTER TABLE main.silver.vave_eventsfeed
CLUSTER BY (h3_ix, event_dt);

%sql
ALTER TABLE main.silver.vave_eventsfeed
SET TBLPROPERTIES (
  'delta.bloomFilter.columns' = 'event_id,h3_ix'
);

# Read only changes since last update
df_changes = spark.readStream.table("main.bronze.vave_eventsfeed") \
    .option("readChangeFeed", "true") \
    .option("startingVersion", last_processed_version)
```

Technical Debt & Future Enhancements

Known Limitations

1. **No Automated Silver Transformation:** Currently manual SQL execution
2. **Limited H3 Neighbor Analysis:** Zone = 1 cell (should include k-ring neighbors)
3. **No Data Quality Constraints:** Missing expectations for lat/lon bounds, risk_score range
4. **Manual Widget Parameters:** Bronze notebook requires manual file path entry
5. **Dashboard Dataset Manual Setup:** Must recreate after workspace migration

Short-Term Enhancements (< 3 months)

- Add data quality checks (expectations on lat/lon, risk_score)
- Create Silver transformation notebook
- Implement H3 k-ring neighbor analysis (1-2 rings)
- Add dashboard date range filters

Medium-Term (3-6 months)

- Streaming migration (if latency requirements change)
- Multi-region support (add region dimension)
- ML model for risk score forecasting
- Interactive geospatial visualization (H3 cell map overlay)

Long-Term (6+ months)

- Real-time alerting (when risk > threshold)
 - Historical trend analysis (multi-year retention)
 - RESTful API for external systems
 - Data mesh architecture (federated ownership)
-